CineVent

Guide d'Installation et de Démarrage

Comment cloner et lancer le projet

Auteurs : Wissal Khalid & Imane El Bouzidi

Année universitaire : 2025/2026
École : ENSA Tétouan

30 mars 2026

Table des matières

1 Prérequis **3**
1.1 Outils obligatoires . . . 3
1.2 Installation du Serverpod CLI . . . 3
1.3 Vérification des installations . . . 3

2 Cloner le projet **3**
2.1 Se positionner dans le dossier de travail . . . 3
2.2 Cloner le repository . . . 4
2.3 Structure du projet cloné . . . 4

3 Créer la base de données **4**

4 Configuration des fichiers **4**
4.1 Créer le fichier passwords.yaml . . . 4
4.2 Vérifier le fichier development.yaml . . . 5

5 Installation des dépendances **5**

6 Appliquer les migrations **5**

7 Correction manuelle de la base de données (si nécessaire) **6**
7.1 Préparation – supprimer les colonnes en trop . . . 6
7.2 Ajouter les colonnes avec le type correct . . . 6
7.3 Rendre les colonnes nullable et créer les index . . . 6
7.4 Corriger la table `favoris` . . . 7
7.5 Vérification . . . 7
7.6 Données de test minimales . . . 7

8 Lancer le serveur backend **8**

9 Lancer l'application Flutter **8**

10 Données de test **9**
10.1 Créer un compte administrateur . . . 11

11 Résolution des problèmes courants **11**
11.1 Erreur : ports déjà utilisés . . . 11
11.2 Erreur : version Serverpod incompatible . . . 12
11.3 Erreur : migration de base de données . . . 12
11.4 Erreur : connexion PostgreSQL refusée . . . 12
11.5 Application Flutter blanche ou noire . . . 12

12 Résumé des commandes **13**

Annexe : Structure des fichiers de configuration **13**

1 Prérequis

Avant de commencer, installez ces outils sur votre machine :

1.1 Outils obligatoires

Outil	Version requise	Lien de téléchargement
Flutter SDK	Version stable	flutter.dev
Dart SDK	Version stable	Inclus avec Flutter
PostgreSQL	14 ou supérieur	postgresql.org
Git	Toute version	git-scm.com
VS Code ou Android Studio	Dernière version	code.visualstudio.com ou developer.android.com/studio

1.2 Installation du Serverpod CLI

Exécutez la commande suivante pour installer Serverpod CLI :

```
dart pub global activate serverpod_cli
```

1.3 Vérification des installations

Ouvrez un terminal et vérifiez que tout est correctement installé :

```
flutter --version
dart --version
psql --version
serverpod --version
```

2 Cloner le projet

2.1 Se positionner dans le dossier de travail

Ouvrez un terminal et naviguez vers votre dossier de projets Flutter :

```
# Windows
cd C:\Users\VOTRE_NOM\StudioProjects

# macOS / Linux
cd ~/StudioProjects
```

2.2 Cloner le repository

```
git clone https://github.com/wissal2002-gif/
   ProjetDevMobileCineReservationEvenement.git
cd ProjetDevMobileCineReservationEvenement
```

2.3 Structure du projet cloné

Après le clonage, vous devriez voir cette structure :

```
ProjetDevMobileCineReservationEvenement/
          cine_reservation_server/   # Backend Serverpod
          cine_reservation_client/   # Client genere (ne pas modifier
   )
          cine_reservation_flutter/  # Application Flutter
          pubspec.yaml               # Workspace Dart
```

3 Créer la base de données

Dans un terminal, exécutez :

```
psql -U postgres -c "CREATE DATABASE cine_reservation;"
```

4 Configuration des fichiers

4.1 Créer le fichier passwords.yaml

Naviguez vers le dossier du serveur et créez le fichier de configuration des mots de passe :

```
cd cine_reservation\cine_reservation_server
code config\passwords.yaml   # Ou ouvrez avec votre   diteur
```

Collez le contenu suivant (remplacez `TON_MOT_DE_PASSE` par votre mot de passe PostgreSQL) :

```
development:
  database: 'TON MOT DE PASSE'
  serviceSecret: 'myServiceSecret12345678901234567890'
  jwtRefreshTokenHashPepper: 'myJwtRefreshTokenHashPepper1234567890'
  jwtHmacSha512PrivateKey: '
     myJwtHmacSha512PrivateKey1234567890abcdefghijklmnopqrstuvwxyz12345
     '
  emailSecretHashPepper: 'myEmailSecretHashPepper1234567890'
```

4.2 Vérifier le fichier development.yaml

Ouvrez `config/development.yaml` et vérifiez que la section `database` est correctement configurée :

```
database:
  host: localhost
  port: 5432
  name: cine_reservation
  user: postgres
```

5 Installation des dépendances

Exécutez les commandes suivantes dans l'ordre :

```
# Server
cd cine_reservation
cd cine_reservation_server
dart pub get

# Client
cd ..\cine_reservation_client
dart pub get

# Flutter
cd ..
cd cine_reservation_flutter
flutter pub get
```

6 Appliquer les migrations

Naviguez vers le dossier du serveur et appliquez les migrations :

```
cd ..
cd cine_reservation_server
dart run bin/main.dart --apply-migrations
```

Résultat attendu :

```
Applied database migration: XXXXXXXXXXXX
Webserver listening on http://localhost:8082
```

Appuyez sur `Ctrl+C` pour arrêter le serveur après la migration.

7 Correction manuelle de la base de données (si nécessaire)

Après avoir appliqué les migrations, il peut arriver que certains avertissements subsistent concernant des colonnes manquantes ou des index. Pour corriger définitivement ces incohérences, exécutez les commandes SQL ci-dessous dans **pgAdmin** (ou via `psql`).

7.1 Préparation – supprimer les colonnes en trop

Si des colonnes `cinemaId` ou `nomCinema` existent déjà, elles risquent d'être mal typées. Commencez par les supprimer proprement (les données sont conservées dans les autres colonnes) :

```sql
-- Supprimer les colonnes probl matiques (si elles existent)
ALTER TABLE films DROP COLUMN IF EXISTS "cinemaId";
ALTER TABLE seances DROP COLUMN IF EXISTS "cinemaId";
ALTER TABLE reservations DROP COLUMN IF EXISTS "cinemaId";
ALTER TABLE utilisateurs DROP COLUMN IF EXISTS "cinemaId";
ALTER TABLE utilisateurs DROP COLUMN IF EXISTS "nomCinema";
```

7.2 Ajouter les colonnes avec le type correct

Recréez les colonnes exactement comme Serverpod les attend (nullable) :

```sql
ALTER TABLE films ADD COLUMN IF NOT EXISTS "cinemaId" integer;
ALTER TABLE seances ADD COLUMN IF NOT EXISTS "cinemaId" integer;
ALTER TABLE reservations ADD COLUMN IF NOT EXISTS "cinemaId" integer;
ALTER TABLE utilisateurs ADD COLUMN IF NOT EXISTS "cinemaId" integer;
ALTER TABLE utilisateurs ADD COLUMN IF NOT EXISTS "nomCinema" text;
```

7.3 Rendre les colonnes nullable et créer les index

Pour que les colonnes acceptent les valeurs `NULL` (conformément aux modèles Dart), et pour créer les index attendus par Serverpod :

```sql
-- Rendre nullable
ALTER TABLE films ALTER COLUMN "cinemaId" DROP NOT NULL;
ALTER TABLE seances ALTER COLUMN "cinemaId" DROP NOT NULL;
ALTER TABLE reservations ALTER COLUMN "cinemaId" DROP NOT NULL;
ALTER TABLE utilisateurs ALTER COLUMN "cinemaId" DROP NOT NULL;
ALTER TABLE utilisateurs ALTER COLUMN "nomCinema" DROP NOT NULL;

-- Index pour films
CREATE INDEX IF NOT EXISTS "film_cinema_idx" ON films ("cinemaId");

-- Index pour seances
```

```sql
DROP INDEX IF EXISTS "seance_cinema_idx";
CREATE INDEX "seance_cinema_idx" ON seances USING btree ("cinemaId");
```

7.4 Corriger la table `favoris`

Le modèle `Favori` attend deux index uniques conditionnels :

```sql
-- Supprimer les anciens index conflictuels
DROP INDEX IF EXISTS "favori_film_idx";
DROP INDEX IF EXISTS "favori_cinema_idx";
DROP INDEX IF EXISTS "favori_film_unique_idx";
DROP INDEX IF EXISTS "favori_cinema_unique_idx";

-- Ajouter la colonne filmId si elle manque
ALTER TABLE favoris ADD COLUMN IF NOT EXISTS "filmId" bigint;
ALTER TABLE favoris ALTER COLUMN "cinemaId" DROP NOT NULL;

-- Cr er les index uniques conditionnels
CREATE UNIQUE INDEX IF NOT EXISTS "favori_utilisateur_film_idx"
ON favoris ("utilisateurId", "filmId") WHERE "filmId" IS NOT NULL;

CREATE UNIQUE INDEX IF NOT EXISTS "favori_utilisateur_cinema_idx"
ON favoris ("utilisateurId", "cinemaId") WHERE "cinemaId" IS NOT NULL
    ;
```

7.5 Vérification

Relancez le serveur après avoir exécuté toutes ces commandes :

```bash
cd cine_reservation_server
dart run bin/main.dart
```

Aucun avertissement ne devrait plus apparaître. Si c'est le cas, redémarrez une dernière fois le serveur pour finaliser.

7.6 Données de test minimales

Pour tester rapidement l'application, insérez ces données de base :

```sql
-- 1. Un cin ma
INSERT INTO cinemas (nom, adresse, ville) VALUES
('CineEvent Casablanca', 'Twin Center, Casablanca', 'Casablanca');

-- 2. Une salle
INSERT INTO salles ("cinemaId", "codeSalle", capacite, "
    typeProjection") VALUES
(1, 'SALLE 1', 100, '2D');

```

```sql
-- 3. Des si ges (rang e A, 10 si ges)
INSERT INTO sieges ("salleId", numero, rangee, type)
SELECT 1, 'A' || i, 'A', 'standard'
FROM generate_series(1, 10) AS i;

-- 4. Un film
INSERT INTO films (titre, synopsis, genre, duree, langue, "cinemaId")
    VALUES
('Interstellar', 'Un voyage interstellaire...', 'Science-Fiction',
   169, 'VF', 1);

-- 5. Une s ance
INSERT INTO seances ("filmId", "salleId", "dateHeure", "prixNormal",
   "placesDisponibles", "typeProjection", "cinemaId")
VALUES (1, 1, NOW() + INTERVAL '2 days', 45.0, 100, '2D', 1);
```

Après insertion, l'application sera prête à être utilisée. Vous pouvez créer un compte utilisateur via l'interface Flutter, puis lui attribuer le rôle `super_admin` pour accéder au panneau d'administration :

```sql
UPDATE utilisateurs SET role = 'super_admin' WHERE email = '
   votre@email.com';
```

8 Lancer le serveur backend

Dans un terminal, démarrez le serveur :

```
cd cine_reservation\cine_reservation_server
dart run bin/main.dart
```

Le serveur est disponible sur :

- **API** : http://localhost:8080
- **Web** : http://localhost:8082

9 Lancer l'application Flutter

Ouvrez un **nouveau terminal** (laissez le serveur tourner dans le premier) :

```
cd cine_reservation\cine_reservation_flutter
flutter run
```

Options de lancement

- **Android** : Connectez un appareil en mode débogage USB ou démarrez un émulateur
- **iOS** (macOS uniquement) : Connectez un appareil ou utilisez le simulateur
- **Web** : Exécutez `flutter run -d chrome` pour lancer dans Chrome

10 Données de test

Pour tester l'application, insérez des données de base dans PostgreSQL :

```sql
-- ============================================================
-- Script d'insertion des utilisateurs - cine_reservation
-- ============================================================

-- Nettoyage pr alable (optionnel, d commentez si besoin)
-- TRUNCATE TABLE utilisateurs RESTART IDENTITY CASCADE;

INSERT INTO utilisateurs (
    id,
    authUserId,
    nom,
    email,
    telephone,
    dateNaissance,
    preferences,
    statut,
    role,
    pointsFidelite,
    photoProfil,
    cinemaId,
    nomCinema
)
VALUES
-- 1. Rachida Hamaa - resp_evenement
(
    2,
    '019ceebb-97ec-7058-9b3c-fa8e3fd543XX',  -- remplacer par l'UUID complet
    'rachidhamaa99',
    'rachidhamaa99@gmail.com',
    '066922095XX',                           -- remplacer par le num ro complet
    '2000-01-01 00:00:00',
    NULL,
    'actif',
```

```sql
    'resp_evenement',
    0,
    NULL,
    NULL,
    NULL
),

-- 2. Ramzi Lkhale - admin_local
(
    4,
    '019d3e22-5099-7b45-a6c9-3dc22b953XX',  -- remplacer par l'UUID complet
    'ramzilkhale',
    'ramzilkhale@gmail.com',
    NULL,
    NULL,
    NULL,
    'actif',
    'admin_local',
    0,
    NULL,
    NULL,
    NULL
),

-- 3. Khalid Abdelhak - super_admin
(
    3,
    '019cf63a-1d73-7563-bc57-b27c82771XX',  -- remplacer par l'UUID complet
    'khalidabdelhak.19...',                  -- remplacer par le nom complet
    'khalidabdelhak.1969@gmail.c...',        -- remplacer par l'email complet
    NULL,
    '2000-01-01 00:00:00',
    NULL,
    'actif',
    'super_admin',
    0,
    NULL,
    NULL,
    NULL
),

-- 4. Amira Amourita - client
(
    1,
```

```sql
    '019ccf66-199b-7ea3-ae06-26d603328fXX',  -- remplacer par l'UUID complet
    'amira amourita',
    'khalid.wissal@etu.uae.ac.ma',
    '0669200949',
    '2000-01-13 00:00:00',
    NULL,
    'actif',
    'client',
    0,
    NULL,
    NULL,
    NULL
)

ON CONFLICT (id) DO UPDATE SET
    authUserId     = EXCLUDED.authUserId,
    nom            = EXCLUDED.nom,
    email          = EXCLUDED.email,
    telephone      = EXCLUDED.telephone,
    dateNaissance  = EXCLUDED.dateNaissance,
    preferences    = EXCLUDED.preferences,
    statut         = EXCLUDED.statut,
    role           = EXCLUDED.role,
    pointsFidelite = EXCLUDED.pointsFidelite,
    photoProfil    = EXCLUDED.photoProfil,
    cinemaId       = EXCLUDED.cinemaId,
    nomCinema      = EXCLUDED.nomCinema;

-- V rification apr s insertion
SELECT id, nom, email, role, statut FROM utilisateurs ORDER BY id;
```

10.1 Créer un compte administrateur

Inscrivez-vous dans l'application via la page d'inscription, puis attribuez le rôle admin dans pgAdmin :

```sql
-- Apres inscription, attribuer le role admin
UPDATE utilisateurs SET role = 'super_admin'
WHERE email = 'votre@email.com';
```

11 Résolution des problèmes courants

11.1 Erreur : ports déjà utilisés

Si vous voyez `Failed to bind socket, port already in use` :

```
# Windows
taskkill /IM dart.exe /F

# macOS / Linux
pkill -f dart
```

Puis relancez le serveur.

11.2 Erreur : version Serverpod incompatible

Assurez-vous que toutes les dépendances Serverpod ont la même version dans les fichiers `pubspec.yaml` (généralement 3.4.x).

11.3 Erreur : migration de base de données

Si `-apply-migrations` échoue :

1. Vérifiez que PostgreSQL est en cours d'exécution
2. Vérifiez les identifiants dans `passwords.yaml` et `development.yaml`
3. Vérifiez que la base de données `cine_reservation` existe

11.4 Erreur : connexion PostgreSQL refusée

— Vérifiez que PostgreSQL est démarré :

```
# Windows
net start postgresql-14

# macOS
brew services start postgresql

# Linux
sudo systemctl start postgresql
```

— Vérifiez que le port 5432 n'est pas bloqué

— Vérifiez que le mot de passe dans `passwords.yaml` est correct

11.5 Application Flutter blanche ou noire

Si l'application s'affiche blanche ou noire :

— Vérifiez que le serveur backend est bien démarré

— Vérifiez la console Flutter pour d'éventuelles erreurs

— Vérifiez que les migrations ont été appliquées avec succès

12 Résumé des commandes

Commande	Description
`git clone [url]`	Cloner le projet depuis GitHub
`cd ProjetDevMobileCineReservationEvenement`	Entrer dans le dossier du projet
`psql -U postgres -c "CREATE DATABASE cine_reservation;"`	Créer la base de données
`dart pub get` (server/client)	Installer les dépendances backend
`flutter pub get` (flutter)	Installer les dépendances Flutter
`dart run bin/main.dart -apply-migrations`	Appliquer les migrations
`dart run bin/main.dart`	Démarrer le serveur backend
`flutter run`	Lancer l'application Flutter
`flutter run -d chrome`	Lancer l'application sur le Web
`flutter doctor`	Diagnostiquer la configuration Flutter

Annexe : Structure des fichiers de configuration

Fichier : `config/passwords.yaml`

```yaml
development:
  database: 'VOTRE_MOT_DE_PASSE_POSTGRESQL'
  serviceSecret: 'myServiceSecret12345678901234567890'
  jwtRefreshTokenHashPepper: 'myJwtRefreshTokenHashPepper1234567890'
  jwtHmacSha512PrivateKey: '
     myJwtHmacSha512PrivateKey1234567890abcdefghijklmnopqrstuvwxyz12345
     '
  emailSecretHashPepper: 'myEmailSecretHashPepper1234567890'
```

Fichier : `config/development.yaml`

```yaml
database:
  host: localhost
  port: 5432
  name: cine_reservation
  user: postgres
```

CineVent - ENSA Tétouan - 2025/2026

Wissal Khalid & Imane El Bouzidi

Projet de développement mobile - Réservation de cinéma et événements